

Square Order Injection QA Hardening Sprint

Date: 2026-07-08

Status: Complete

Summary

Square order injection is hardened for broader beta use with improved QA visibility, retry safety, total/payment comparison, failure handling, and operator clarity. Checkout, payouts, Deliverect routing, and manual/tablet routing are unchanged. No Square status webhooks were added.

`orderRoutingMode=square` remains the single vendor-level control;

`squareOrderRoutingEnabled` is deprecated/ignored.

Files changed

Area	Files
QA runbook	<code>docs/integrations/square-order-injection-qa.md</code> (new)
Total comparison	<code>src/lib/integrations/square/square-order-total-comparison.ts</code> , <code>.test.ts</code>
Audit helpers	<code>src/lib/integrations/square/square-order-audit.ts</code> , <code>.test.ts</code>
Square submit / retry	<code>src/services/square-order.service.ts</code> , <code>.test.ts</code>
Audit types	<code>src/lib/integrations/square/square-order.types.ts</code>
Env vars	<code>src/lib/env.ts</code>
Retry eligibility	<code>src/lib/admin-needs-attention-actions.ts</code> , <code>admin-needs-attention-actions.square.test.ts</code>
Admin order detail	<code>src/lib/admin-order-detail-query.ts</code> , <code>AdminDeliverectDiagnosticsPanel.tsx</code>
Admin debug endpoint	<code>src/app/admin/(dashboard)/orders/[orderId]/square-routing-debug/route.ts</code> , <code>square-order-routing-debug.server.ts</code>
Vendor messaging	<code>src/lib/vendor-order-vendor-display.ts</code> , <code>.test.ts</code> , <code>VendorOrdersLedger.tsx</code> , <code>VendorOrdersLedgerRow.tsx</code> , <code>VendorOrderDetailPanel.tsx</code>
Docs	<code>docs/ops/order-routing-env-vars.md</code> , <code>docs/integrations/square-production-checklist.md</code> , <code>docs/reports/square-order-injection.md</code>

Part 1 — QA runbook

Added `docs/integrations/square-order-injection-qa.md` with a 10-step sandbox test plan covering:

1. Environment confirmation (`ENABLE_SQUARE_INTEGRATION` , `SQUARE_ROUTING_LIVE` , `SQUARE_ENVIRONMENT` , OAuth scopes)
 2. Square sandbox connection
 3. Catalog import
 4. Menu preview/publish
 5. `orderRoutingMode=square`
 6. Public orderability
 7. Customer order variants (single item, multi-qty, modifiers, multi-item, notes)
 8. OO-side verification (Stripe, `routingStatus` , `squareOrderId` , audit, no false success issues)
 9. Square-side verification (order, pickup, prepaid/external, source, line items, no re-collection)
 10. Payout regression check (unchanged)
-

Part 2 — Total/payment comparison

Behavior

On successful Square `CreateOrder` , the service reads `order.total_money.amount` . On `CreatePayment` with `EXTERNAL` , it stores the payment amount used. These are compared to the OO vendor food total (`subtotalCents` + `taxCents`).

Field (audit JSON)	Source
<code>ooSubtotalCents</code>	VendorOrder
<code>ooTaxCents</code>	VendorOrder
<code>ooTotalCents</code>	Computed
<code>squareOrderTotalCents</code>	CreateOrder response
<code>squareExternalPaymentCents</code>	CreatePayment request
<code>squareTotalDifferenceCents</code>	<code>abs(ooTotalCents - squareOrderTotalCents)</code>

Stored under `lastSquarePayload.reconciliation` (no new Prisma columns).

Config

Variable	Default	Behavior
<code>SQUARE_TOTAL_MISMATCH_WARN_CENTS</code>	<code>1</code>	Admin warning when difference $\geq$ threshold
<code>SQUARE_TOTAL_MISMATCH_BLOCK_CENTS</code>	unset	Optional hard block when difference $\geq$ threshold

Admin copy: "Square calculated a different order total than Open Order. Verify taxes, modifiers, discounts, or catalog prices before relying on Square totals for reconciliation."

OO/Stripe remains source of truth for customer payment and payouts.

Part 3 — Retry safety and reconciliation

Idempotency keys (stable per vendor order)

- Order: `oo:sq:order:{vendorOrderId}`
- Payment: `oo:sq:pay:{vendorOrderId}`

Cases handled

Case	Behavior
CreateOrder failed before Square order ID	Retry reuses order idempotency key
CreateOrder succeeded, local save failed	Partial persist of <code>squareOrderId</code> + audit before payment
CreateOrder succeeded, EXTERNAL payment failed	No duplicate CreateOrder; payment-only retry on existing Square order
Timeout after CreateOrder	Same idempotency key; Square returns same order when safe
Duplicate retry after success	Skipped when <code>routingStatus=sent</code> and <code>squareOrderId</code> present

Retry button eligibility

Shown only when `canRetryRouting()` returns true for Square:

- Vendor `orderRoutingMode=square`
- Square connection healthy with required scopes
- `SQUARE_ROUTING_LIVE=true`
- Mappings still valid

- Not already `sent` with Square order ID
- Payment-only retry allowed when `squareOrderId` exists but routing not `sent`

Admin copy: "Retry Square routing uses Square idempotency keys to avoid duplicate Square orders."

Part 4 — Persisted routing audit state

Uses existing `VendorOrder.lastSquarePayload` / `lastSquareResponse` JSON (no secrets or OAuth tokens).

`SquareOrderSubmitAudit` now includes:

- `squareOrderId` (column)
 - `squarePaymentId` , `squareOrderState` , `squarePaymentStatus`
 - `squareSubmittedAt` , `squareLastAttemptAt` (columns / audit)
 - `squareLastError` (column)
 - `createOrderPayload` / `createOrderResponse`
 - `createPaymentPayload` / `createPaymentResponse`
 - `reconciliation` (total comparison)
 - `paymentOnlyRetry` flag when retry skips CreateOrder
-

Part 5 — Admin order detail clarity

Square-routed orders in admin order detail show:

- Provider: Square
- Routing status, Square order ID, payment ID/status
- Submitted at / last attempted at / attempt count
- Total comparison with mismatch warning
- Contextual failure guidance:
 - Missing scopes → reconnect ORDERS_WRITE/PAYMENTS_WRITE
 - `SQUARE_ROUTING_LIVE=false` → global kill switch message
 - Missing mappings → re-import/publish guidance
- Retry Square routing when eligible
- Collapsible payload/response audit

Debug JSON: `GET /admin/orders/{orderId}/square-routing-debug` (admin-only, no secrets).

Part 6 — Vendor dashboard clarity

Square vendors see plain-English status (no raw payloads):

State	Copy
Success (sent)	"Sent to Square"
Failure	"Square routing failed — Open Order still has the paid order"
Pending	"Sending to Square"

No Deliverect/POS lock language for Square vendors. Customer payment is not implied to have failed when only Square routing failed.

Part 7 — Failure-mode tests

#	Scenario	Coverage
1	Success path (CreateOrder + EXTERNAL payment, IDs saved)	<code>square-order.service.test.ts</code>
2	Missing ORDERS_WRITE scope + reconnect guidance	<code>square-order.service.test.ts</code>
3	Global kill switch (<code>SQUARE_ROUTING_LIVE=false</code>)	<code>square-order-routing-readiness.test.ts</code>
4	Missing item/modifier mapping blocks routing	<code>square-order-mapper.test.ts</code>
5	CreateOrder API failure → routing issue	<code>square-order.service.test.ts</code>
6	Payment failure after CreateOrder → partial persist, payment-only retry	<code>square-order.service.test.ts</code>
7	Duplicate retry does not duplicate Square order	<code>square-order.service.test.ts</code> , <code>admin-needs-attention-actions.square.test.ts</code>
8	Total mismatch warning (≥1¢ default)	<code>square-order-total-comparison.test.ts</code>
9	Block threshold (optional env)	Logic in <code>evaluateSquareOrderTotalComparison</code> ; block disabled by default
10	Cross-provider regression	<code>routing.service.test.ts</code> , <code>provider-ux-regression.test.ts</code>

112 Square/routing-related unit tests passing. `npm run build` succeeds.

Part 8 — Admin debug endpoint

GET /admin/orders/{orderId}/square-routing-debug returns safe JSON:

- Routing mode, env flags (`SQUARE_ROUTING_LIVE` , mismatch thresholds)
- Connection health and scopes
- Mapping coverage summary
- Square order/payment IDs and statuses
- Total comparison
- Last error
- Retry eligibility

No secrets or tokens.

Acceptance criteria

Criterion	Status
Successful injection records order ID, payment ID, status, audit	✓
Square order appears prepaid/external	✓ (unchanged EXTERNAL payment flow)
Admin sees total comparison and retry eligibility	✓
Retry does not create duplicate Square orders	✓
Failures create routing issues; paid OO order preserved	✓
Vendors see simple status, not diagnostics	✓
Checkout, payouts, Deliverect, manual/tablet unchanged	✓

Known limitations

1. **No Square webhooks** — one-way injection; Square fulfillment state is not synced back to OO.
2. **Total comparison is advisory by default** — OO/Stripe charge may differ from Square catalog-calculated total; block threshold is opt-in.
3. **Reconciliation uses food subtotal + tax** — does not compare Stripe all-in customer total (fees/tips excluded by design).

4. **squareOrderRoutingEnabled deprecated** — only `orderRoutingMode=square` + readiness gates matter.
 5. **Order-level debug endpoint has no dedicated route test yet** — vendor-level debug route is tested; order route follows same loader pattern.
-

Cross-provider regression

- **Manual/tablet routing:** unchanged (`routing.service.test.ts`)
- **Deliverect routing:** unchanged (`routing.service.test.ts` , provider UX regression)
- **Stripe checkout / payouts:** untouched in this sprint
- **No silent fallback** to manual or Deliverect on Square failure